## What is Apache Airflow ?

**Apache Airflow** is an open-source workflow orchestration tool used to **schedule, automate, and monitor data pipelines and workflows** programmatically using Python. It helps manage task dependencies, execution order, and workflow scheduling.

It is maintained by the Apache Software Foundation.

## Key concepts

- **DAG (Directed Acyclic Graph)**
  A workflow defined as a series of tasks with dependencies.
  Example: download data → clean data → load into database
- **Tasks**
  Individual units of work (e.g., run a script, query a database)
- **Scheduler**
  Decides when to run tasks based on timing and dependencies
- **Executor**
  Handles how tasks are executed (locally, distributed, etc.)
- **Web UI**
  Lets you monitor workflows, see logs, retry failed tasks

## How it works

1. You define workflows in **Python code**
2. Airflow reads and schedules them
3. Tasks run in the correct order
4. You track everything in a dashboard

#### Example use case

A daily data pipeline:

- Extract data from an API
- Transform it (clean/aggregate)
- Load it into a data warehouse
- Send a notification if something fails

Airflow ensures each step runs in sequence and on schedule.

## Advantages of Apache Airflow

1. **Workflow Automation**
   Automates complex workflows and data pipelines without manual intervention.
2. **Easy Scheduling**
   Allows tasks to run at specific times (daily, hourly, weekly, etc.).
3. **Python-Based**
   Workflows are written in Python, making them flexible and easy to customize.
4. **Task Dependency Management**
   Ensures tasks run in the correct order based on dependencies.
5. **Monitoring and Logging**
   Provides a web UI to track workflow status, logs, retries, and failures.
6. **Scalability**
   Can handle small workflows as well as large enterprise-level pipelines.
7. **Retry and Failure Handling**
   Automatically retries failed tasks and supports alert notifications.
8. **Integration Support**
   Connects with databases, cloud platforms, Spark, Hive, Hadoop, and many other tools.
9. **Open Source**
   Free to use with strong community support from the Apache Software Foundation.
10. **Dynamic Pipeline Creation**
    Pipelines can be created dynamically using code instead of manual configuration.

# Apache Airflow Architecture

Apache Airflow architecture consists of multiple components that work together to schedule, execute, and monitor workflows.

**Main Components**

1. **Web Server**
- Provides the Airflow UI
- Used to monitor DAGs, task status, logs, and workflow execution
2. **Scheduler**
- Core component of Airflow
- Checks DAGs and schedules tasks based on time and dependencies
3. **Metadata Database**
- Stores DAG information, task states, logs, users, and schedules
- Commonly uses MySQL or PostgreSQL
4. **Executor**
- Executes tasks assigned by the scheduler
- Types include Local Executor, Celery Executor, and Kubernetes Executor
5. **Workers**
- Execute the actual tasks in distributed environments
- Mainly used with Celery Executor
6. **DAG Directory**
- Contains Python files defining workflows (DAGs)
- Scheduler continuously scans this folder

## Workflow Execution Flow

1. User creates a DAG in Python
2. DAG is stored in the DAG folder
3. Scheduler reads the DAG and schedules tasks
4. Executor sends tasks to workers
5. Workers execute tasks
6. Metadata Database stores execution details
7. Web Server displays status and logs to users

## Simple Definition

Apache Airflow architecture is a combination of scheduler, executor, workers, metadata database, and web server that together manage workflow orchestration and task execution.

## What is DAG

In Apache Airflow, a DAG (Directed Acyclic Graph) is a workflow that defines a sequence of tasks and their dependencies.

## Definition

A DAG is a collection of tasks organized in a way that:

- Directed → Tasks have a defined execution order
- Acyclic → No task loops back to itself
- Graph → Tasks are connected like a flowchart

---

#### Example

Task A → Task B → Task C

Here:

- Task B starts only after Task A finishes
- Task C starts only after Task B finishes

---

### Purpose of DAG

- Organizes workflows
- Manages task dependencies
- Schedules and automates pipelines

---

## In Airflow

DAGs are written in Python files and stored in the DAG folder. The scheduler reads them and executes tasks in the defined order.

## What are Executors ?

In Apache Airflow, **Executors** are components responsible for deciding **how and where tasks are executed** after the scheduler triggers them.

## Definition

An Executor is the mechanism that runs Airflow tasks by assigning them to workers or systems for execution.

---

## Types of Executors in Airflow

### 1. Sequential Executor

- Runs one task at a time
- Mainly used for testing or development

### 2. Local Executor

- Runs multiple tasks in parallel on a single machine
- Suitable for small to medium workloads

### 3. Celery Executor

- Distributed executor using Celery workers
- Supports multiple worker machines
- Good for large-scale production systems

### 4. Kubernetes Executor

- Runs each task inside a separate Kubernetes pod
- Provides high scalability and isolation

### 5. Debug Executor

- Used for debugging DAGs during development

---

## Working of Executor

1. Scheduler identifies tasks to run
2. Executor receives tasks from scheduler
3. Executor sends tasks to workers or execution environments
4. Tasks are executed and status is updated in the metadata database

---

## Simple Definition

Executors in Airflow are components that control task execution and determine where and how workflow tasks run.